

Dynamic Agent Knowledge Graphs via LLM & GraphRAG

Executive Brief: Mapping Agent Epistemic Boundaries, Belief Evolution, and Cross-Agent Knowledge Contagion

Core Answer: Yes, absolutely. By combining Large Language Models with Graph Retrieval-Augmented Generation (GraphRAG) and temporal event logs, organizations can construct a living, visual topology that explicitly maps (1) what specific epistemic domains an agent commands, (2) which knowledge clusters it draws from, and (3) how inter-agent negotiations, critique loops, and information exchanges structurally reshape those boundaries over time.

1. Technical Architecture: From Unstructured Agent Memory to Attributed Graphs

Traditional RAG relies on isolated vector similarity (cosine distance over chunked text), creating disconnected knowledge silos. By integrating an LLM extraction pipeline with graph structures (e.g., Neo4j, NetworkX, Memgraph), every agent's internal memory, tool outputs, and incoming messages are converted into a dynamic semantic network:

Layer	Mechanism (LLM + GraphRAG)	What It Visualizes & Exposes
Entity-Relation Extraction	LLM ingests agent system prompts, private vector DBs, and execution traces; extracts (<i>Subject</i> , <i>PREDICATE</i> , <i>Object</i>) triples.	Domain Boundaries: Highlights agent specialty clusters (e.g., Legal Constraints vs. Code Generation vs. Market Risk).
Hierarchical Community Detection	Algorithms (e.g., Leiden/Louvain) group dense subgraphs into macro/micro knowledge sectors with LLM-generated summaries.	Topic Heatmaps: Visual clusters color-coded by owning agent, showing overlap, blind spots, and domain redundancy.
Temporal Knowledge Versioning	Every node, relation edge, and property is assigned a temporal validity window: [<i>t_created</i> , <i>t_updated</i> , <i>source_event_id</i>].	State Trajectory: Enables time-slider replay to observe how an agent's understanding evolves turn-by-turn.

2. Modeling Cross-Agent Dynamics: 4 Interaction Patterns

When agents communicate (in peer-to-peer debates, supervisor-worker delegating, or critic-curator loops), their epistemic state changes through four concrete graph operations:

Interaction Type	Graph Mutation Operation	Observable Structural Impact
Knowledge Injection	Agent B receives verified facts from Agent A's tool execution.	New Node/Edge Infusion: Agent B's sub-graph sprouts foreign nodes linked back to Agent A with attribution provenance.
Belief Revision / Disproof	Critic agent challenges an assumption with counter-evidence.	Edge Pruning / Weight Degradation: Prior confidence weights on relations decrease, or edges are flagged as 'Refuted'.
Consensus & Edge Bridging	Two specialists align on an interdisciplinary trade-off.	Cross-Cluster Bridges: Thick semantic bridges form between formerly disjoint subgraphs (e.g., Cost Model <--> Security).
Epistemic Contagion & Drift	Hallucination or bias propagates across multi-agent turns.	Contagion Propagation: Graph diff tool visually flashes infected downstream nodes, enabling root-cause audit.

3. Practical Stack & Business Value

Recommended Open Implementation Stack:

- **Extraction & Community Summarization:** Microsoft GraphRAG / LlamaIndex PropertyGraph / LangGraph.
- **Graph Store & Querying:** Neo4j (Cypher) or KuzuDB (embedded, high-performance local graph).
- **Interactive UI / Front-end:** WebGL/Canvas-based force-directed graphs (Cosmograph, Cytoscape.js, 3d-force-graph).

Why It Matters: Moving from black-box LLM context windows to an explicit Knowledge Graph gives engineering and compliance teams complete observability: you can visually trace *who told what to whom*, quantify each agent's domain authority, and immediately detect dangerous knowledge drift before autonomous actions execute.